

Nome: _____ Matrícula: _____

Instruções. Todas as questões são discursivas. Apresente o **desenvolvimento completo** de cada resposta. Respostas sem justificativa não serão consideradas. A soma das questões é **exatamente 10,0 pontos**.

Questão 1 — Tabela Hash com Encadeamento (2,0 pts)

A Defesa Civil cadastra pluviômetros instalados na cidade por código numérico em uma **tabela hash com encadeamento** de tamanho $m = 7$, usando a função hash por divisão $h(k) = k \bmod 7$. Os seguintes códigos chegam nessa ordem e devem ser inseridos **no início** da lista de cada slot:

3, 10, 27, 34, 8, 15, 22.

- (a) Execute as inserções passo a passo, calculando $h(k)$ para cada chave. Mostre o estado final da tabela (listas em cada slot) e identifique todas as colisões.
- (b) Calcule o **fator de carga** $\alpha = n/m$ após todas as inserções. Supondo hashing uniforme simples, qual é o custo *esperado* de uma busca **sem sucesso**? Justifique a partir da análise de hashing uniforme simples.
- (c) A equipe técnica sugere trocar para $m = 11$ (primo). Calcule a nova função hash $h'(k) = k \bmod 11$ para cada uma das sete chaves e mostre o novo estado da tabela. Compare o fator de carga e o comprimento máximo das listas obtidos com $m = 7$ e com $m = 11$. Por que se recomenda escolher m primo e distante de potências de 2?

Questão 2 — Endereçamento Aberto com Hashing Duplo (2,0 pts)

O serviço de transporte público rastreia identificadores de veículos da frota de ônibus em uma tabela de tamanho $m = 11$ usando **endereçamento aberto com hashing duplo**:

$$h_1(k) = k \bmod 11, \quad h_2(k) = 1 + (k \bmod 7), \quad h(k, i) = (h_1(k) + i h_2(k)) \bmod 11.$$

Os seguintes identificadores chegam em ordem: 5, 16, 8, 19, 12, 27.

- (a) Para cada chave, calcule h_1 , h_2 e rastreie a **sequência de sondagem** até encontrar um slot vazio. Apresente o estado final da tabela após todas as inserções.
- (b) Mostre que, ao contrário da sondagem linear, dois IDs distintos com o mesmo h_1 seguem passos diferentes. Identifique quais chaves deste exemplo compartilham o mesmo h_1 e verifique que suas sequências de sondagem divergem. Nomeie o tipo de agrupamento (*primário* ou *secundário*) que essa propriedade elimina e explique por que a sondagem linear sofre de **ambos** os tipos.
- (c) Calcule α após as seis inserções. Usando a fórmula $\frac{1}{1-\alpha}$ (hashing uniforme, endereçamento aberto), estime o número médio de sondagens em uma busca **sem sucesso**. Por que $h_2(k)$ deve ser coprimo com m ? Verifique isso para os valores usados nesta questão.

Questão 3 — Árvore AVL: Inserção e Rotações (2,0 pts)

A biblioteca pública organiza os códigos de suas publicações em uma **árvore AVL** inicialmente vazia. Os seguintes valores são inseridos **nesta ordem**:

10, 20, 30, 25, 5, 27.

Utilize a convenção de que a **altura de um nó** é o número de arestas no caminho mais longo até uma folha (folha tem altura 0, subárvore vazia tem altura -1); o **fator de balanceamento** de um nó v é $\text{bf}(v) = h(\text{sub-árvore esquerda de } v) - h(\text{sub-árvore direita de } v)$.

- (a) Insira as seis chaves uma a uma. Após cada inserção, atualize o fator de balanceamento de cada nó no caminho de retorno à raiz. Identifique **se e onde** ocorre violação ($|\text{bf}| > 1$) e classifique o caso (Esq-Esq, Dir-Dir, Esq-Dir ou Dir-Esq).
- (b) Aplique a rotação necessária em cada violação e desenhe a **árvore final** após todas as inserções, indicando o fator de balanceamento de cada nó.
- (c) Remova a chave **20** da árvore resultante do item (b) usando o método padrão de remoção AVL (substitua pelo *sucessor in-order* quando necessário). Se houver desbalanceamento, identifique o caso e aplique a rotação correspondente. Qual é a diferença crítica entre remoção e inserção em árvores AVL quanto ao número de rotações necessárias?

Questão 4 — Heap Binário e Fila de Prioridade**(2,0 pts)**

O SAMU usa um **min-heap** para triagem de ocorrências durante a temporada de chuvas: chave menor = prioridade maior (ocorrência mais crítica atendida primeiro). O array inicial de prioridades das ocorrências é:

$$A = [8, 3, 7, 1, 5, 4], \quad \text{índices } 0, 1, 2, 3, 4, 5.$$

(Fórmulas do heap em base 0: filhos de i em $2i + 1$ e $2i + 2$; pai de i em $\lfloor (i - 1)/2 \rfloor$.)

- (a) Execute o **build-heap** aplicando *sift-down* de baixo para cima, começando no último nó interno. Mostre o estado do array após cada *sift-down*. Por que o build-heap custa $O(n)$ e não $O(n \log n)$?
- (b) A partir do heap construído no item (a), execute **duas operações** EXTRACT-MIN consecutivas, mostrando o estado do array e da árvore após cada extração. Qual elemento é extraído em cada chamada?
- (c) Descreva como o **Heap Sort** usa o max-heap para ordenar A **in-place**. Aplique as duas primeiras trocas (raiz $\leftrightarrow$ fim) com os respectivos *sift-downs* no array $A = [8, 3, 7, 1, 5, 4]$, construindo primeiro o max-heap. Qual é a complexidade de tempo no pior caso e por que o Heap Sort não supera o Quick Sort na prática?

Questão 5 — Conjuntos Disjuntos e Union-Find**(2,0 pts)**

A Defesa Civil agrupa sete bairros em zonas de resposta a emergências. Numeramos os bairros de 1 a 7: 1 = Centro, 2 = Bairro Norte, 3 = Bairro Sul, 4 = Jardins, 5 = Bairro Leste, 6 = Bairro Oeste, 7 = Industrial.

Após MAKE-SET para cada bairro, executam-se as seguintes uniões usando a representação por **floresta** com **union por rank** (*em caso de empate de rank, o primeiro argumento torna-se raiz e seu rank é incrementado*):

$$\text{UNION}(1, 2), \quad \text{UNION}(3, 4), \quad \text{UNION}(5, 6), \quad \text{UNION}(1, 3), \quad \text{UNION}(5, 7).$$

- (a) Execute cada UNION acima. Após cada operação, mostre os arrays *parent*[1..7] e *rank*[1..7]. Desenhe a floresta resultante após o último UNION.
 - (b) Execute **FIND(4)** com **compressão de caminho**. Trace o caminho percorrido do bairro 4 até a raiz. Mostre o array *parent* depois da compressão. Por que o rank **não** é atualizado durante a compressão? (*Dica: o rank é um limitante superior da altura, não a altura exata; argumente que a compressão jamais aumenta a altura real, portanto o invariante continua válido.*)
 - (c) Execute **UNION(2,6)**: determine as raízes de cada argumento (use o *parent* resultante do item (b)) e aplique union por rank. Quantos conjuntos disjuntos existem ao final? Justifique por que uma sequência genérica de m operações sobre n elementos roda em tempo total $O(m \alpha(n))$ amortizado, onde α é a **inversa da função de Ackermann**, e não em $O(m \log n)$ — o que as duas heurísticas contribuem individualmente e o que sua combinação garante?
-